Large Language Models: Foundations, Architecture, Training, and Evaluation

Sepehr Kerachi

Copyright © 2026 Sepehr Kerachi. All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without prior written permission of the copyright holder, except as permitted by law.

Edition: First Edition

Contents

I Foundations and Architecture **1**

1 Introduction **2**

2 Architecture **3**
2.1 Tokenization . . . 3
2.1.1 From Written Text to Token Sequences . . . 3
2.1.2 From Word-Level to Subword-Level Representations . . . 5
2.1.3 Balancing Tokens and Types: The Principle of Least Effort . . . 6
2.1.4 A Stochastic Framework for Tokenization . . . 9
2.1.5 Statistical and Computational Properties . . . 12
2.1.6 Limitations and Connection to Positional Encoding . . . 14
2.2 Positional Encoding . . . 15
2.2.1 Why Self-Attention Requires Positional Information . . . 15
2.2.2 Absolute Positional Encoding . . . 17
2.2.3 Combining Content and Position: Addition and Concatenation . . . 19
2.2.4 Relative Positional Encoding and Rotary Position Embedding . . . 20
2.2.5 Long-Context Position Rescaling and Multi-Scale Positional Encoding 21
2.2.6 Limitations and Connection to the Next Architectural Components . 25
2.3 Feed-Forward Networks . . . 25
2.3.1 Position-Wise Feed-Forward Networks . . . 27
2.3.2 Parameterization and Computational Complexity . . . 28
2.3.3 Gated Feed-Forward Networks and Their Internal Units . . . 30
2.3.4 Feed-Forward Networks Within Transformer Blocks . . . 31
2.3.5 From Dense FFNs to Mixture-of-Experts Layers . . . 32
2.3.6 Switch Routing, Expert Capacity, and Load Balancing . . . 33
2.3.7 Fine-Grained Segmentation and Shared Experts . . . 35
2.3.8 Expert Structure, Specialization, and Practical Constraints . . . 38
2.3.9 Limitations and Connection to Activation Functions . . . 39
2.4 Activation Functions . . . 39
2.5 Normalization . . . 39
2.6 Attention Mechanisms . . . 39

II Optimization and Scaling **40**

3 Optimization **41**

3.1 Optimization Algorithms . . . 41
3.2 Loss Functions . . . 41
3.3 Quantization . . . 41
3.4 Knowledge Distillation . . . 41
3.5 Parameter-Efficient Fine-Tuning . . . 41

4 Scaling Laws **42**

III Reinforcement Learning and Neuroscience **43**

5 Reinforcement Learning **44**
5.1 Theoretical Foundations . . . 44
5.2 Actor–Critic Methods . . . 44
5.3 Reinforcement Learning with Verifiable Rewards . . . 44
5.4 Group Relative Policy Optimization for Verifiable Rewards . . . 44

6 Neuroscience and Cognitive Foundations **45**
6.1 Introduction . . . 45
6.2 From System 1 to System 2 . . . 45

IV Data and Training **46**

7 Training Data **47**
7.1 Pre-Training Data . . . 47
7.2 Post-Training Data . . . 47

8 Training Methods **48**
8.1 Pre-Training . . . 48
8.2 Post-Training . . . 48
8.3 Machine Unlearning . . . 48

V Evaluation, Impact, Philosophy, and Challenges **49**

9 Evaluation **50**

10 Environmental Impacts **51**

11 Philosophy **52**

12 Challenges **53**

Part I

Foundations and Architecture

1. Introduction

2. Architecture

2.1 Tokenization

The architecture of a language model begins with a representational question: what are the discrete units over which the model will operate? A neural network does not receive written language as an unstructured sequence of words in their ordinary typographical form. It receives a numerical representation of a sequence of discrete units. Tokenization constructs this representation by mapping character strings into sequences drawn from a vocabulary. The choice of these units affects the length of the sequences processed by the model, the size of its vocabulary, the linguistic regularities available to it, and the probability distribution that it learns to estimate. Tokenization is therefore not merely an incidental preprocessing operation; it is the first representational decision in the architecture considered in this chapter.

Two complementary perspectives are needed to understand this decision. The cognitive perspective examines how the number and nature of linguistic units affect the effort required to process language. In particular, it considers the trade-off between the number of token occurrences and the number of distinct vocabulary types. The formal perspective treats a tokenizer as a pair of mappings between character strings and token sequences and asks whether these mappings preserve the statistical and computational properties required by language modeling. The former perspective motivates flexible vocabularies containing units of different sizes, while the latter establishes conditions under which a model trained over token sequences remains a sound estimator of a distribution over original texts (**yang2024rethinking**; **gastaldi2025foundations**).

2.1.1 From Written Text to Token Sequences

A formal description of tokenization begins by distinguishing the elementary symbols of a text from the units selected for a model's vocabulary. Let Σ be a finite, nonempty alphabet of elementary symbols. Depending on the textual representation, these symbols may be characters or other elementary textual units. For each nonnegative integer n, let Σ^n denote the set of strings containing exactly n symbols from Σ. The set of all finite strings over the alphabet is the Kleene closure

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n. \tag{2.1}$$

It includes the empty string, denoted by ε_Σ. The length of a string $\sigma \in \Sigma^*$ is written as $|\sigma|$. If $\sigma_1, \sigma_2 \in \Sigma^*$, their concatenation is written as $\sigma_1 \cdot \sigma_2$. Concatenation is associative and has the empty string as its identity:

$$(\sigma_1 \cdot \sigma_2) \cdot \sigma_3 = \sigma_1 \cdot (\sigma_2 \cdot \sigma_3), \qquad \varepsilon_\Sigma \cdot \sigma = \sigma \cdot \varepsilon_\Sigma = \sigma. \tag{2.2}$$

Thus, Σ^*, equipped with concatenation and the empty string, is a free monoid. This structure is useful because it describes strings without imposing a particular linguistic segmentation on them (**gastaldi2025foundations**).

A tokenizer introduces a second set, denoted by Δ, whose elements are the units in the tokenizer's vocabulary. In contrast to Σ, whose elements are elementary symbols, Δ may contain units of different lengths. A vocabulary may include individual characters, subword units, complete words, and, in some designs, multiword expressions. The set of all finite sequences over this vocabulary is

$$\Delta^* = \bigcup_{n=0}^{\infty} \Delta^n. \tag{2.3}$$

An element of Σ^* is a character string or text, whereas an element of Δ^* is a token sequence. The distinction is mathematical rather than hierarchical: the formal framework does not assume that characters are intrinsically more fundamental than tokens. It distinguishes the two sets because tokenization studies the mappings between them (**gastaldi2025foundations**).

It is also necessary to distinguish a token from a token type. A token is an occurrence of a unit in a particular tokenized text. A token type is a distinct unit that can occur in the vocabulary. If a corpus contains the word *language* several times, each occurrence contributes a token, while the word itself contributes only one type. Consequently, the number of tokens measures the length of the corpus after segmentation, whereas the number of types measures the size of the set of distinct units used to represent it.

Let a corpus be

$$\mathcal{C} = \{\sigma_1, \sigma_2, \ldots, \sigma_M\},$$

where each $\sigma_m \in \Sigma^*$, and let T be a deterministic tokenizer. The total number of token occurrences produced by T is

$$N_T(\mathcal{C}) = \sum_{m=1}^{M} |T(\sigma_m)|. \tag{2.4}$$

If $\mathrm{Types}(T(\sigma_m))$ denotes the set of distinct token types appearing in the tokenization of σ_m, the number of types observed in the corpus is

$$V_T(\mathcal{C}) = \left| \bigcup_{m=1}^{M} \mathrm{Types}(T(\sigma_m)) \right|. \tag{2.5}$$

These quantities describe different properties of a representation. Reducing $N_T(\mathcal{C})$ does not necessarily reduce $V_T(\mathcal{C})$, and reducing the vocabulary size does not necessarily shorten the tokenized corpus. The tension between these objectives is central to tokenizer design (**yang2024rethinking**).

2.1.2 From Word-Level to Subword-Level Representations

Word-level tokenization treats complete words as the primary units. In writing systems that use spaces to separate words, a tokenizer can use spaces and punctuation as practical segmentation cues. This representation is intuitive because its units often correspond to familiar lexical items. However, the vocabulary required to cover a language is not fixed. Morphological inflections, newly coined words, spelling variation, and domain-specific terminology continually introduce forms that may not have appeared in the training corpus. A word-level tokenizer must either add these forms to its vocabulary or handle them as out-of-vocabulary (OOV) items. The problem is particularly visible in languages whose writing systems do not mark word boundaries with spaces, because segmentation cannot be obtained from whitespace alone (**yang2024rethinking**).

Character-level tokenization takes the opposite approach. It represents a text as a sequence of elementary characters. This substantially limits the number of distinct vocabulary units and allows an unfamiliar word to be represented whenever its constituent characters are available. Its cost is sequence length: a word that would be one token under a word-level representation may require several character tokens. Longer sequences require the language model to process and predict more units. Character-level tokenization therefore reduces vocabulary complexity while increasing the number of token occurrences.

Subword tokenization occupies an intermediate position. It constructs units that may be larger than individual characters but smaller than complete words. Frequently occurring character sequences can be represented as single vocabulary entries, while rare or unseen words can be decomposed into smaller units. The result is an open-vocabulary strategy: the tokenizer does not need a separate entry for every possible complete word, provided that the relevant elementary symbols and subword components can be represented.

Byte Pair Encoding (BPE) is a prominent example. Its underlying pair-merging procedure originated in data compression and was adapted to subword representation. In its basic form, the corpus is initialized as sequences of small units. The algorithm repeatedly identifies frequent adjacent pairs and merges selected pairs into larger units. Repeating the operation builds a vocabulary containing units of different lengths. Frequent sequences can then be represented compactly, while less frequent forms remain expressible through smaller components. WordPiece is another subword approach. Its maximal-munch encoding procedure repeatedly selects the longest vocabulary prefix matching the remaining input. Unigram tokenization can represent a text through alternative subword segmentations; when regularization is used, different segmentations can be sampled during training. Yang also discusses SentencePiece as a language-independent subword framework, including its relevance to languages that do not use spaces as word boundaries (**yang2024rethinking**; **gastaldi2025foundations**).

The principal distinction among these approaches is not simply the length of the resulting units. Their segmentation procedures determine which sequences are valid representations, whether the encoder is deterministic or stochastic, and how many alternative token sequences may correspond to the same text. These differences become important when the tokenizer is treated as part of a probabilistic language model rather than as a stand-alone text-processing utility.

Subword segmentation also does not resolve every linguistic representation problem. Multiword expressions (MWEs) are expressions composed of multiple words whose con-

ventional meaning may not be reducible to the literal meanings of their components. The expression *kick the bucket*, for example, has an idiomatic meaning that differs from its literal composition. If such an expression is represented only through its constituent words or subwords, the model must learn its conventional meaning from contextual examples. Adding an MWE as a single vocabulary unit may make its holistic meaning more directly available, but it can also increase the number of types. Rare or domain-specific MWEs may not occur often enough in training data to support a reliable representation. Yang therefore treats direct MWE representation as a vocabulary-design trade-off rather than as a rule that every multiword expression should become a token (**yang2024rethinking**).

2.1.3 Balancing Tokens and Types: The Principle of Least Effort

The difference between token count and vocabulary size can be quantified. Yang reports a comparison on a German corpus in which word-level segmentation produces 100 million tokens and $1{,}750{,}000$ types, BPE produces 111 million tokens and $82{,}000$ types, and character-level segmentation produces 550 million tokens and $3{,}000$ types. These values are reproduced in table 2.1.

Table 2.1: Token and type counts for three segmentation methods on the German corpus reported by Yang.

Segmentation method	Number of tokens	Number of types
Words	100 million	$1{,}750{,}000$
BPE	111 million	$82{,}000$
Characters	550 million	$3{,}000$

Source: **yang2024rethinking**, reproducing the comparison attributed there to the BPE literature.

The BPE vocabulary contains

$$\frac{82{,}000}{1{,}750{,}000} \approx 0.0469 \tag{2.6}$$

of the word-level types, or approximately 4.7%. This corresponds to a reduction of approximately 95.3% in the number of types. Its token count, however, is

$$\frac{111}{100} = 1.11 \tag{2.7}$$

times the word-level count, an increase of approximately 11%. Character-level segmentation reduces the vocabulary to approximately 0.17% of the word-level type count, but increases the number of tokens to 5.5 times the word-level count. The comparison illustrates the central compromise: word-level tokenization uses relatively few tokens but requires a large vocabulary; character-level tokenization uses a small vocabulary but creates long sequences; subword tokenization can sharply reduce the vocabulary while increasing the token count much less severely (**yang2024rethinking**).

The trade-off is affected by the language being represented. In morphologically rich languages, subword units can capture recurring components across inflected forms, reducing the need to store every form independently. In Chinese, the absence of whitespace

word boundaries and the prevalence of compounds create different segmentation challenges. Yang notes that Chinese BPE units are often individual characters and reports that a Chinese sentence may require approximately 1.7 times as many tokens as a comparable English sentence, while Burmese or Amharic may require as many as ten times the number. These observations indicate that a tokenizer's efficiency depends not only on its algorithm but also on the writing system and statistical structure of the corpus (**yang2024rethinking**).

Yang interprets the token–type trade-off through the Principle of Least Effort (PLE). In this account, language processing involves both immediate and future effort. Fewer token occurrences reduce the burden associated with maintaining and processing units during immediate comprehension; Yang relates this burden to working memory. Fewer distinct types reduce the burden of storing and retrieving different units; this is associated with long-term memory. The two objectives can conflict: merging units into longer tokens may shorten a sequence but introduce additional vocabulary types, while removing types may require a text to be represented through more tokens.

The relationship also has a qualitative connection to Minimum Description Length (MDL). In an MDL interpretation, fewer types correspond to a more compact description of the lexicon or encoder model, whereas fewer tokens correspond to a more compact description of the encoded corpus. Yang emphasizes an important distinction: the cognitive account is not simply a single global minimization of the sum of token count and type count. It treats the minimization of the two burdens as distinct processes whose interaction produces a balance. The corresponding design objectives can be written schematically as

$$\min_T N_T(\mathcal{C}) \qquad \text{and} \qquad \min_T V_T(\mathcal{C}), \tag{2.8}$$

with the understanding that these objectives are generally competing rather than independently attainable (**yang2024rethinking**).

The Less-is-Better model. The Less-is-Better (LiB) model is Yang's proposed implementation of PLE. Its vocabulary is not restricted to a single linguistic level: it can contain subwords, words, and MWEs. The model has two interacting mechanisms, called the Memorizer and the Forgetter. It initially divides the input corpus into small units. The Memorizer repeatedly merges adjacent units into longer units and stores the resulting units in the vocabulary. Longer units can reduce the number of tokens required to represent a text, but every newly stored unit can increase the number of types. The Forgetter removes less useful vocabulary entries, including infrequent types and types whose use increases the number of tokens needed for sentences. Removing types reduces vocabulary size but can require the corresponding text to be represented by smaller units. The two mechanisms therefore act in opposite directions and are intended to approach a relatively stable vocabulary (**yang2024rethinking**).

The LiB paper identifies larger-first selection and counterfactual evaluation as segmentation strategies, and memorizing, re-ranking, and forgetting as vocabulary-update strategies. It does not provide a fully parameterized optimization objective, a universal update schedule, or a numerical convergence threshold in the paper considered here. The mechanism should therefore be understood as the proposed learning-and-forgetting design, not as a completely specified algorithm with a uniquely determined implementa-

tion. The interaction between corpus segmentation and vocabulary updating is summarized in figure 2.1.

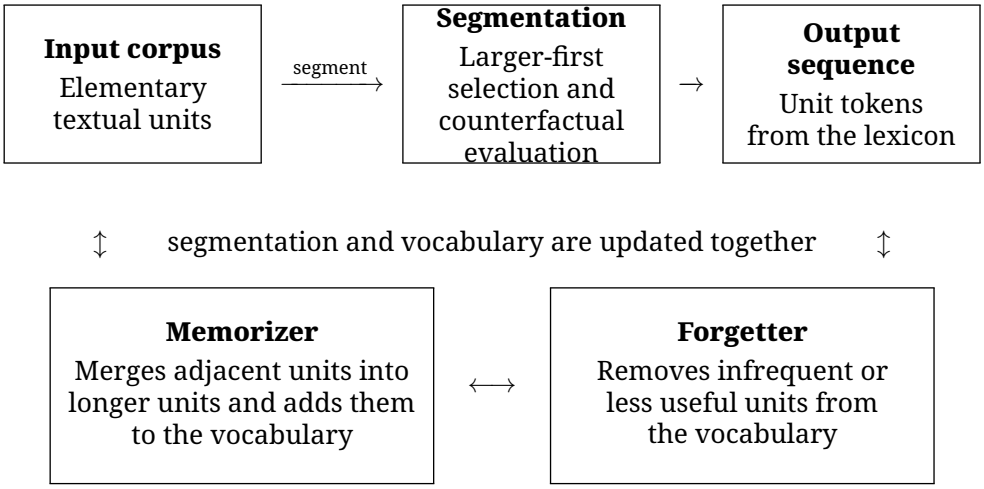


Figure 2.1: Conceptual flow of the Less-is-Better tokenizer. The Memorizer introduces longer units, which can reduce token count while increasing vocabulary size. The Forgetter removes less useful types, reducing vocabulary size while potentially increasing the number of tokens needed for a text. Their interaction is intended to approach a relatively stable vocabulary. Original schematic based on the mechanism described by **yang2024rethinking**.

Yang reports LiB segmentations that combine units of different sizes. For example, the English input ‘allrightwhydon’tweputhimawaynow‘ is represented as ‘allright | whydon’t | we | puthimaway | now‘, rather than as the nine word-level units ‘all | right | why | don’t | we | put | him | away | now‘. The example illustrates that LiB does not impose a strict boundary between subwords, words, and multiword expressions. It searches for units that function usefully within the corpus under its learning-and-forgetting procedure (**yang2024rethinking**).

The reported vocabulary and average-token-length measurements are shown in table 2.2. The BRphono average length is measured in English phonemes, while the CTB8 average length is measured in Chinese characters. The results show that LiB learns longer average units than the other listed methods in both corpora. Its vocabulary size is smaller than BPE’s in BRphono, but larger than the word vocabulary; in CTB8, its vocabulary is smaller than the word vocabulary but larger than the BPE vocabulary. Thus, the measurements support a different balance between unit length and vocabulary size, rather than a claim that LiB has the smallest vocabulary under every comparison.

The paper also reports bits-per-character (BPC) scores from simple 2-gram and 3-gram language-model evaluations. If x is an evaluated character sequence containing N_c characters and $p_\theta(x)$ is the probability assigned to it by a model, its description length per character in bits is

$$\mathrm{BPC}(x) = -\frac{1}{N_c} \log_2 p_\theta(x). \tag{2.9}$$

A lower score indicates a shorter model-based description of the evaluated sequence. The reported results are reproduced in table 2.3.

Table 2.2: Average token length and vocabulary size reported for BRphono and CTB8.

Corpus	Measure	Characters	BPE	Words	LiB
BRphono (English)	Average token length	1.0	2.8	2.9	3.6
BRphono (English)	Vocabulary size	50	5,574	1,321	1,869
CTB8 (Chinese)	Average token length	1.0	1.4	1.7	1.9
CTB8 (Chinese)	Vocabulary size	4,697	7,980	65,410	39,320

Source: **yang2024rethinking**. Average length is measured in English phonemes for BRphono and Chinese characters for CTB8.

Table 2.3: Bits-per-character scores reported for 2-gram and 3-gram models under different tokenizations.

Corpus	Model	Characters	BPE	Words	LiB
CTB8	2-gram	3.558	2.788	2.333	**2.095**
CTB8	3-gram	2.025	1.193	1.163	**0.903**
BRphono	2-gram	2.221	1.003	0.977	**0.791**
BRphono	3-gram	1.371	0.563	0.584	**0.484**

Source: **yang2024rethinking**.

LiB obtains the lowest reported BPC in each of these four comparisons. The scope of the result must nevertheless be kept precise. Yang describes LiB as a cognitive model that was not optimized specifically for large language models; the reported language-model evaluations use simple 2-gram and 3-gram models. The results therefore motivate further investigation of PLE-based tokenizers, but they do not establish that LiB outperforms established tokenizers in large-scale transformer language models (**yang2024rethinking**).

2.1.4 A Stochastic Framework for Tokenization

The token–type trade-off concerns the efficiency and linguistic suitability of a representation. A separate question is whether the representation preserves the probability distribution of the original text. A language model is intended to estimate probabilities of linguistic sequences, but after tokenization it estimates probabilities over token sequences. To understand the relationship between these two distributions, Gastaldi et al. formulate tokenization through stochastic maps (**gastaldi2025foundations**).

A stochastic map from a countable set X to a countable set Y, written $f : X \rightsquigarrow Y$, assigns a probability distribution over Y to every input $x \in X$. Its transition probabilities satisfy

$$f(y \mid x) \geq 0, \qquad \sum_{y \in Y} f(y \mid x) = 1 \quad \text{for every } x \in X. \tag{2.10}$$

A deterministic function is a special case in which all probability mass for an input is concentrated on one output. The stochastic-map formulation therefore includes deterministic tokenizers as well as tokenizers that permit alternative segmentations.

Definition 2.1 **Tokenizer model.** A tokenizer model from Σ^* to Δ^* is a pair

$$T = (\tau, \kappa),$$

where

$$\tau : \Sigma^* \rightsquigarrow \Delta^* \qquad \text{and} \qquad \kappa : \Delta^* \rightsquigarrow \Sigma^*$$

are stochastic maps. The map τ is the encoder and the map κ is the decoder.

The encoder and decoder are defined as composable mappings between the two string spaces. No additional relationship between the alphabet Σ and the vocabulary Δ is assumed by the general definition. This breadth is deliberate: it allows the statistical consequences of the complete encoding–decoding process to be studied independently of the particular segmentation algorithm.

Let p^* be a reference probability distribution over Σ^*. Applying the encoder produces a probability distribution over token sequences, called the image or pushforward distribution:

$$q^* = \tau p^*. \tag{2.11}$$

For an individual token sequence $\delta \in \Delta^*$, this means

$$q^*(\delta) = \sum_{\sigma \in \Sigma^*} \tau(\delta \mid \sigma) p^*(\sigma). \tag{2.12}$$

The language model estimates q^* from tokenized data. Let $\{q_n\}$ be a sequence of estimators, where n indexes the amount of information used for estimation. If the estimators are consistent, then q_n converges to q^*. Decoding the estimated token distribution gives

$$p_n = \kappa q_n, \qquad p_n(\sigma) = \sum_{\delta \in \Delta^*} \kappa(\sigma \mid \delta) q_n(\delta). \tag{2.13}$$

The target of interest is whether p_n converges to the original distribution p^*, not merely whether q_n converges to the token-level distribution q^*.

The key mathematical property is that stochastic maps preserve consistency of estimators on the countable spaces considered by Gastaldi et al. In particular, if probability distributions p_n converge pointwise to a probability distribution p, then their L^1 distance converges to zero:

$$\lim_{n \to \infty} \sum_{x \in X} |p_n(x) - p(x)| = 0. \tag{2.14}$$

For a stochastic map $f : X \rightsquigarrow Y$, the triangle inequality and the normalization in equation (2.10) give

$$\begin{aligned} \sum_{y \in Y} |(fp_n)(y) - (fp)(y)| &= \sum_{y \in Y} \left| \sum_{x \in X} f(y \mid x)(p_n(x) - p(x)) \right| \\ &\leq \sum_{y \in Y} \sum_{x \in X} f(y \mid x) |p_n(x) - p(x)| \\ &= \sum_{x \in X} |p_n(x) - p(x)| \sum_{y \in Y} f(y \mid x) \\ &= \sum_{x \in X} |p_n(x) - p(x)|. \end{aligned} \tag{2.15}$$

Thus, applying a stochastic map cannot increase the L^1 distance between the distributions. This contraction property explains why consistency is preserved when an estimator is transformed through a stochastic map (**gastaldi2025foundations**).

Theorem 2.1 **Fundamental Principle of Tokenization.** Let p^* be a reference probability distribution over Σ^*, let $T = (\tau, \kappa)$ be a tokenizer, and let $\{q_n\}$ be a consistent sequence of estimators of $q^* = \tau p^*$. Then the decoded estimators $\{\kappa q_n\}$ are consistent estimators of p^* if and only if

$$\boxed{\kappa\tau p^* = p^*.} \tag{2.16}$$

Proof. Because $\{q_n\}$ is consistent for q^*, preservation of consistency under stochastic maps gives

$$\kappa q_n \longrightarrow \kappa q^*.$$

By the definition $q^* = \tau p^*$, the limiting distribution is

$$\kappa q^* = \kappa\tau p^*.$$

The decoded estimators therefore converge to p^* exactly when their limiting distribution equals p^*, which is the condition $\kappa\tau p^* = p^*$. This proves both directions. □

The theorem separates two kinds of correctness. A language model may estimate the token-level distribution consistently, yet the resulting character-level estimator may still be inconsistent if the tokenizer changes the reference distribution. The condition in equation (2.16) is therefore a property of the tokenizer together with the target distribution; it is not guaranteed solely by successful optimization of the language model.

Definition 2.2 **Consistency and exactness.** A tokenizer $T = (\tau, \kappa)$ is consistent with respect to a probability distribution p over Σ^* if

$$\kappa\tau p = p.$$

It is exact if

$$\kappa\tau = \mathrm{id}_{\Sigma^*}, \tag{2.17}$$

where id_{Σ^*} is the identity map on character strings.

Exactness implies consistency for every probability distribution over Σ^*. The converse does not hold if consistency is established only for one particular distribution: two strings with equal probability may have their probability assignments interchanged without changing that distribution. Gastaldi et al. establish that exactness is equivalent to consistency with respect to every probability distribution over Σ^*. They also show that, for an exact tokenizer, the decoder is deterministic on the support of the encoder. In the corresponding sense, the encoder is injective: distinct original texts cannot assign positive encoding probability to a common token sequence. This is the formal reason that lossless recovery and statistical consistency are connected (**gastaldi2025foundations**).

The two spaces and the statistical requirement are summarized in figure 2.2. The diagram makes explicit that the model estimates a distribution in token space and that the decoder must return this estimate to the original string space without changing the target distribution.

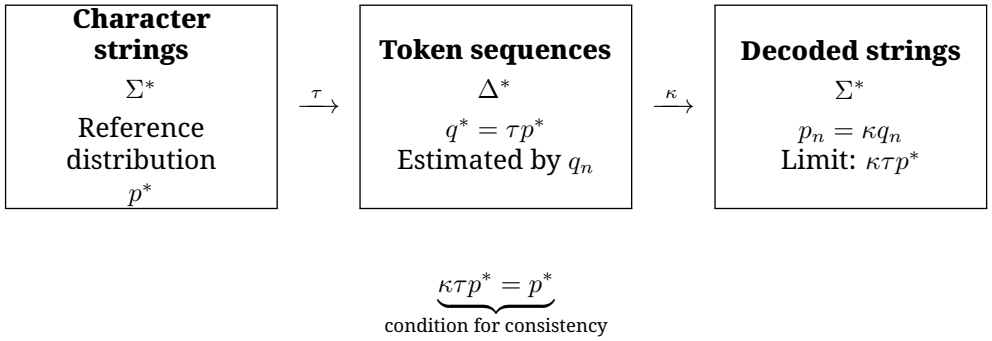


Figure 2.2: The statistical path induced by a tokenizer. The encoder maps the reference distribution over character strings to a distribution over token sequences; the language model estimates the token-level distribution; and the decoder maps that estimate back to character space. Consistency with the original distribution requires the encoder–decoder composition to preserve p^*, as established by **gastaldi2025foundations**.

2.1.5 Statistical and Computational Properties

The formal conditions above identify failure modes that can arise in actual tokenizers. Gastaldi et al. organize these concerns into statistical problems, especially inconsistency and ambiguity, and computational problems, especially finiteness and sequentiality (**gastaldi2025foundations**).

Noninjective encoding and inconsistency. An encoder is noninjective when distinct character strings can be mapped to the same token sequence. For example, preprocessing operations such as lowercasing, stripping accents, removing punctuation, or standardizing whitespace can cause distinct original strings to receive identical representations. If the decoder does not retain the information removed by preprocessing, the original string cannot in general be recovered. The composition $\kappa\tau$ is then not the identity, and the consistency condition may fail.

The use of an unknown symbol can create a related problem. If multiple out-of-vocabulary symbols or strings are mapped to one distinguished unknown representation, their probability mass is combined. A decoder that has no information distinguishing the original inputs cannot generally reconstruct their separate probabilities. Including the elementary alphabet in the vocabulary can support open-vocabulary encoding, but out-of-alphabet symbols at training or inference time still require careful treatment. The relevant requirement is not merely that an input can be assigned a token; it is that the complete encoding–decoding process preserves the distinctions needed for the target distribution (**gastaldi2025foundations**).

Noninjective decoding and ambiguity. A decoder is noninjective when more than one token sequence maps to the same character string. For example, if the vocabulary contains the units `t`, `h`, `e`, and `he`, the text `the` can be represented as `t | h | e` or as `t | he`. A deterministic encoder may select only one representation, while the decoder accepts both. If a language model assigns positive probability to token sequences outside the image of the encoder, the decoded text distribution can receive probability from al-

ternative sequences. Gastaldi et al. call this effect spurious ambiguity when it arises from estimation rather than from the reference tokenization process.

Softmax-based neural models are relevant here. For finite logits $z_1, \ldots, z_K$, the softmax probability of token i is

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}. \tag{2.18}$$

Every such probability is strictly positive. Consequently, a neural model may assign positive probability to token sequences that a deterministic encoder would never produce. This can reintroduce ambiguity even when the tokenizer itself is bijective over the encoder's image.

Ambiguity can also be deliberate. Stochastic tokenization and subword regularization expose a model to multiple segmentations of the same text during training. The intended benefit is to improve the handling of compositional structure, noise, and tokenization variation. Spurious ambiguity and intentional stochastic ambiguity have different origins, but both require the probability of a character string to account for all token sequences that decode to it. For a deterministic decoder, this marginal probability is

$$p_n(\sigma) = \sum_{\delta:\, \kappa(\delta)=\sigma} q_n(\delta). \tag{2.19}$$

For a stochastic decoder, the corresponding expression is the sum in equation (2.13). The sum can be computationally demanding when the set of preimages is large or infinite (**gastaldi2025foundations**).

Multiplicativity and finite preimages. A decoder is multiplicative if it respects concatenation. Writing token-sequence concatenation as $\odot$, the condition is

$$\kappa(\delta' \odot \delta'') = \kappa(\delta') \cdot \kappa(\delta''). \tag{2.20}$$

The commonly used BPE, WordPiece, and Unigram tokenizers discussed by Gastaldi et al. have multiplicative decoders. A multiplicative decoder has a trivial kernel if it maps every nonempty token sequence to a nonempty character string:

$$\delta \neq \varepsilon_\Delta \implies \kappa(\delta) \neq \varepsilon_\Sigma. \tag{2.21}$$

A sufficient condition is that no individual vocabulary token is erased during decoding.

Under multiplicativity and a trivial kernel, each token contributes at least one character to the decoded text. Therefore,

$$\kappa(\delta) = \sigma \implies |\delta| \leq |\sigma|. \tag{2.22}$$

If $|\sigma| = n$, every token sequence that decodes to σ has length at most n. For a finite vocabulary, the number of possible sequences of these lengths is bounded by

$$|\kappa^{-1}(\sigma)| \leq \sum_{i=1}^{n} |\Delta|^i, \tag{2.23}$$

for nonempty σ. Thus, the set of preimages is finite. The bound is exponential in the length of the text, however, so finiteness alone does not guarantee an efficient marginalization algorithm. Gastaldi et al. note that the desired computational complexity of the

complete composition $\kappa\tau$ should ideally be at most linear in the input length. For exact deterministic tokenizers such as BPE and WordPiece, the composition can be computed in linear time when the encoder and decoder themselves admit linear-time implementations, although the rigorous treatment of spurious ambiguity remains a separate challenge (**gastaldi2025foundations**).

Prefix monotonicity and sequential generation. A decoder has prefix monotonicity when decoding any token prefix yields a prefix of the complete decoded text:

$$\delta' \preceq \delta \implies \kappa(\delta') \preceq \kappa(\delta). \tag{2.24}$$

Here, $\preceq$ denotes the prefix relation. Multiplicativity implies prefix monotonicity under the framework considered by Gastaldi et al. This property matters for autoregressive language models because they generate token sequences incrementally. A prefix-monotone decoder preserves the relationship between a generated token prefix and the corresponding character prefix.

Bounded variation and sequentiality. For strings $\gamma, \gamma' \in \Gamma^*$, let $\gamma \wedge \gamma'$ be their longest common prefix. The left distance between the strings is

$$d_{\mathrm{L}}(\gamma, \gamma') = |\gamma| + |\gamma'| - 2|\gamma \wedge \gamma'|. \tag{2.25}$$

A function $f : \Gamma^* \to B^*$ has bounded variation if, for every $k \geq 0$, there is a finite constant $C_k \geq 0$ such that

$$d_{\mathrm{L}}(\gamma, \gamma') \leq k \implies d_{\mathrm{L}}(f(\gamma), f(\gamma')) \leq C_k. \tag{2.26}$$

In other words, inputs that differ by a bounded amount outside their longest common prefix cannot produce outputs whose corresponding difference is arbitrarily large.

Gastaldi et al. connect bounded variation with subsequential functions from formal language theory. Under the additional condition that the function preserves rational sets, bounded variation implies that the function is subsequential. A subsequential function can be implemented by a deterministic finite-state transducer with an additional output function for terminal states. Since subsequential functions are closed under composition, the encoder and decoder can be studied through the finite-state realizability of their individual mappings.

WordPiece's maximal-munch procedure repeatedly selects the longest vocabulary prefix that matches the remaining input. The maximum length of vocabulary units bounds this prefix search, supporting a subsequential realization. Gastaldi et al. also discuss results that construct deterministic finite automata for BPE encoders under specific conditions on the merge rules. These connections show that a tokenizer's practical suitability depends not only on the number of tokens it produces but also on whether its mappings can be implemented through tractable sequential computation (**gastaldi2025foundations**).

2.1.6 Limitations and Connection to Positional Encoding

The two perspectives developed in this section address different but complementary requirements. The Principle of Least Effort motivates a vocabulary that balances the number of token occurrences against the number of distinct types. The LiB model illustrates

how memorization and forgetting can construct units of different sizes, including subwords, words, and multiword expressions. Its reported 2-gram and 3-gram results are encouraging within the scope of the experiments, but they do not constitute a large-scale transformer evaluation. The numerical evidence should therefore be treated as motivation for further research rather than as a universal ranking of tokenizer algorithms (**yang2024rethinking**).

The formal framework establishes a separate condition: a consistent estimator over token sequences yields a consistent estimator over original strings only when the tokenizer's encoder–decoder composition preserves the target distribution. Noninjective encoding can destroy distinctions between texts; noninjective decoding can create ambiguity; and marginalizing over alternative token sequences can be computationally expensive. Multiplicativity, a trivial decoder kernel, prefix monotonicity, and bounded variation provide formal tools for analyzing finiteness and sequential implementation (**gastaldi2025foundations**).

Tokenization consequently determines the discrete units and sequence length on which the remaining architecture operates, but it does not itself specify how the model represents the positions or order of those units. Once a text has been converted into a token sequence, the architecture must provide a representation of where each token occurs in that sequence. This is the next representational problem: positional encoding supplies information about token positions and their relationships, allowing subsequent computational layers to process not only token identities but also their order.

2.2 Positional Encoding

Once tokenization has converted an input text into a sequence of discrete tokens, the Transformer must construct numerical representations that retain the information needed for subsequent computation. Token embeddings represent the identities and learned content of the tokens, but a sequence is not defined by token identity alone. The order in which tokens occur, and the distances separating them, are also essential to linguistic structure. The next architectural question is therefore how a Transformer can represent sequence position and make that information available to its attention mechanism.

This requirement follows from the computational organization of the Transformer. Unlike recurrent architectures, which propagate information through an explicitly ordered sequence of state updates, self-attention computes interactions among token representations through query, key, and value projections. This design enables parallel processing and direct interactions between distant tokens, but the attention operation does not, by itself, provide a complete representation of the positions occupied by those tokens. Positional encoding supplies this missing structural information. The distinction between token content and token position, and the way in which they are combined, consequently form part of the mathematical definition of a Transformer layer.

2.2.1 Why Self-Attention Requires Positional Information

Let an input sequence contain L tokens, and let the embedding of the token at position i be $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$. Collecting the embeddings as rows gives

$$X = \begin{bmatrix} \mathbf{x}_1^\mathsf{T} \\ \mathbf{x}_2^\mathsf{T} \\ \vdots \\ \mathbf{x}_L^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}. \tag{2.27}$$

For one attention head, the query, key, and value matrices are obtained through learned linear projections,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{2.28}$$

where $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. The query and key dimension is d_k, and the value dimension is d_v. The scaled dot-product attention scores and attention weights are

$$S = \frac{QK^\mathsf{T}}{\sqrt{d_k}}, \qquad A = \text{softmax}_{\text{row}}(S), \qquad Z = AV. \tag{2.29}$$

The row-wise softmax normalizes each query's scores over the keys that it is permitted to attend to. In unmasked self-attention, each row of A sums to one. The factor $\sqrt{d_k}$ controls the scale of the dot products as the query and key dimension changes. Without this scaling, the variance of the dot products can grow with d_k, pushing softmax toward saturated distributions and making optimization less well-conditioned. The formulation in equation (2.29) is the self-attention basis from which the positional mechanisms considered in this section are developed (**zhou2024rethinking**).

The limitation of content-only self-attention can be made precise through permutation equivariance. Let $P \in \mathbb{R}^{L \times L}$ be a permutation matrix. Left multiplication by P rearranges the rows of a sequence matrix. If the input is permuted, then

$$X' = PX, \qquad Q' = PQ, \qquad K' = PK, \qquad V' = PV. \tag{2.30}$$

For unmasked self-attention, the score matrix becomes

$$S' = \frac{Q'K'^\mathsf{T}}{\sqrt{d_k}} \tag{2.31}$$

$$= \frac{(PQ)(PK)^\mathsf{T}}{\sqrt{d_k}} \tag{2.32}$$

$$= P\frac{QK^\mathsf{T}}{\sqrt{d_k}}P^\mathsf{T} \tag{2.33}$$

$$= PSP^\mathsf{T}. \tag{2.34}$$

Because softmax is applied independently to each row, permuting both the query rows and key columns permutes the attention matrix in the same way: $A' = PAP^\mathsf{T}$. The output is therefore

$$Z' = A'V' \tag{2.35}$$
$$= PAP^{\mathsf{T}}PV \tag{2.36}$$
$$= PAV \tag{2.37}$$
$$= PZ. \tag{2.38}$$

Thus, content-only, unmasked self-attention is permutation-equivariant: permuting the input representations permutes the outputs, but the operation does not independently encode the original sequential positions. In an autoregressive language model, a causal mask additionally restricts each query to preceding positions. The mask establishes a direction of information flow, but positional representations are still needed to provide the attention computation with a more expressive account of absolute positions and relative displacements. Positional encoding addresses this requirement by making position-dependent information available to the network.

2.2.2 Absolute Positional Encoding

An absolute positional encoding assigns a representation to each position independently of the positions occupied by other tokens. If the token at position p has content embedding $\mathbf{x}_p$, its positional representation is denoted by $\mathbf{e}_p$. In the conventional additive formulation, the input to the subsequent Transformer computation is

$$\mathbf{h}_p = \mathbf{x}_p + \mathbf{e}_p, \qquad \mathbf{x}_p, \mathbf{e}_p \in \mathbb{R}^{d_{\text{model}}}. \tag{2.39}$$

The positional vector must have the same dimensionality as the token embedding for this addition to be defined. Absolute positional representations can be generated by predetermined functions or learned as model parameters. The two alternatives considered by **zhou2024rethinking** are sinusoidal positional encoding and learnable absolute positional encoding.

Sinusoidal positional encoding. Sinusoidal positional encoding constructs each positional vector from sine and cosine functions with different frequencies. For an embedding dimension d_{model}, the components are

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2.40}$$
$$\text{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2.41}$$

where p is the position index and i indexes a pair of embedding dimensions. Defining the angular frequency

$$\omega_i = 10000^{-2i/d_{\text{model}}}, \tag{2.42}$$

the two components associated with frequency ω_i can be written as

$$\mathbf{p}_i(p) = \begin{bmatrix} \sin(p\omega_i) \\ \cos(p\omega_i) \end{bmatrix}. \tag{2.43}$$

The frequencies decrease geometrically as the pair index increases. The lower-index pairs therefore change more rapidly with position, while the higher-index pairs change more slowly. The complete positional vector combines these frequency scales, giving the model a deterministic multiscale representation of position.

For example, when $d_{\text{model}} = 4$, there are two frequency pairs. Their frequencies are

$$\omega_0 = 10000^0 = 1, \qquad \omega_1 = 10000^{-1/2} = 0.01. \tag{2.44}$$

At position $p = 2$, the positional vector is

$$\text{PE}(2) = \begin{bmatrix} \sin(2) \\ \cos(2) \\ \sin(0.02) \\ \cos(0.02) \end{bmatrix} \tag{2.45}$$

$$\approx \begin{bmatrix} 0.9093 \\ -0.4161 \\ 0.0200 \\ 0.9998 \end{bmatrix}. \tag{2.46}$$

The first pair varies substantially over a small positional change, whereas the second pair varies slowly. Their combination allows positions to be represented through multiple periodic scales rather than through a single monotonically increasing coordinate.

A useful property of this construction is that a fixed positional displacement can be represented by a linear transformation. For a fixed offset k, the trigonometric addition identities give

$$\mathbf{p}_i(p+k) = \begin{bmatrix} \cos(k\omega_i) & \sin(k\omega_i) \\ -\sin(k\omega_i) & \cos(k\omega_i) \end{bmatrix} \mathbf{p}_i(p). \tag{2.47}$$

The transformation in equation (2.47) depends on the offset k, not on the absolute position p. Although the encoding is defined using absolute positions, this property gives a model a mathematical route to learning relationships associated with relative displacements. The sinusoidal construction is deterministic and does not require a separately learned positional parameter vector for every position (**zhou2024rethinking**).

Learnable absolute positional encoding. An alternative is to learn a positional vector for each position. For a maximum represented sequence length L_{max}, the positional parameters can be arranged as

$$E_{\text{pos}} \in \mathbb{R}^{L_{\text{max}} \times d_{\text{model}}}, \qquad \mathbf{e}_p = E_{\text{pos}}[p, :]. \tag{2.48}$$

The token and positional vectors are then combined using equation (2.39). The number of positional parameters is

$$N_{\text{pos}} = L_{\text{max}} d_{\text{model}}. \tag{2.49}$$

Unlike the sinusoidal method, the learned table does not impose a predetermined frequency structure across positions. Its positional vectors are optimized with the other model parameters. This flexibility makes the representation data-dependent, while the

explicit table also ties the learned parameters to the positions represented during training. The two methods therefore differ in how positional structure is supplied: sinusoidal encoding imposes a fixed multiscale construction, whereas learnable encoding estimates the positional vectors through optimization.

2.2.3 Combining Content and Position: Addition and Concatenation

The construction of a positional vector is only one part of the design. The model must also determine how positional information is combined with token content before the attention projections are applied. Addition is the conventional method, but concatenation provides an alternative that keeps the two components in distinct parts of the representation.

Under addition, the query and key vectors at positions i and j are

$$\mathbf{q}_i = (\mathbf{x}_i + \mathbf{p}_i)W_Q, \qquad \mathbf{k}_j = (\mathbf{x}_j + \mathbf{p}_j)W_K. \tag{2.50}$$

Let $W_{QK} = W_Q W_K^\mathsf{T}$. The unnormalized dot product can be expanded into four interactions:

$$\begin{aligned}\mathbf{q}_i\mathbf{k}_j^\mathsf{T} &= (\mathbf{x}_i + \mathbf{p}_i)W_{QK}(\mathbf{x}_j + \mathbf{p}_j)^\mathsf{T} \\ &= \mathbf{x}_i W_{QK}\mathbf{x}_j^\mathsf{T} + \mathbf{p}_i W_{QK}\mathbf{x}_j^\mathsf{T} \\ &\quad + \mathbf{x}_i W_{QK}\mathbf{p}_j^\mathsf{T} + \mathbf{p}_i W_{QK}\mathbf{p}_j^\mathsf{T}.\end{aligned} \tag{2.51}$$

The first term is a content-to-content interaction. The second is a position-to-content interaction, the third is a content-to-position interaction, and the fourth is a position-to-position interaction. In the additive formulation, these interactions share the same learned query-key interaction matrix W_{QK}. They are consequently not parameterized as four independent transformations.

zhou2024rethinking use this decomposition to motivate a concatenation-based alternative. To maintain the same total input dimension, the content and positional components are each assigned half of the representation dimension:

$$\mathbf{x}'_i, \mathbf{p}'_i \in \mathbb{R}^{d_{\text{model}}/2}, \qquad \mathbf{h}'_i = \mathbf{x}'_i \oplus \mathbf{p}'_i \in \mathbb{R}^{d_{\text{model}}}, \tag{2.52}$$

where $\oplus$ denotes vector concatenation. The concatenated representation is projected into queries and keys in the usual way. If the resulting query-key interaction matrix is partitioned into four blocks,

$$W_{QK} = \begin{bmatrix} W_{cc} & W_{cp} \\ W_{pc} & W_{pp} \end{bmatrix}, \tag{2.53}$$

then the score contains separately parameterized content-to-content, content-to-position, position-to-content, and position-to-position interactions:

$$\begin{aligned}\mathbf{q}'_i\mathbf{k}'^\mathsf{T}_j &= \mathbf{x}'_i W_{cc}\mathbf{x}'^\mathsf{T}_j + \mathbf{x}'_i W_{cp}\mathbf{p}'^\mathsf{T}_j \\ &\quad + \mathbf{p}'_i W_{pc}\mathbf{x}'^\mathsf{T}_j + \mathbf{p}'_i W_{pp}\mathbf{p}'^\mathsf{T}_j.\end{aligned} \tag{2.54}$$

The block decomposition makes the distinction between the two methods explicit. Addition uses a shared interaction matrix for the four terms, whereas concatenation allows the corresponding blocks to be learned independently. The dimension reduction in equation (2.52) keeps the concatenated input dimension equal to that of the additive representation. The conceptual difference is illustrated in figure 2.3.

Zhou et al. argue that content and positional vectors represent distinct types of information and that independently parameterized interaction spaces can therefore be beneficial. Their experiments compare addition and concatenation in several single-scale and multiscale Vision Transformer models. They report performance differences ranging from approximately 0.1 to 0.5 percentage points in top-1 ImageNet-1K accuracy in favor of concatenation for several evaluated models, without a systematic increase in computation overhead. These are results from vision architectures and image-classification experiments; they should not be interpreted as direct evidence that concatenation is superior for language models. The contribution relevant to language-model architecture is the analysis of how the choice of combination operation changes the parameterization of content-position interactions (**zhou2024rethinking**).

2.2.4 Relative Positional Encoding and Rotary Position Embedding

Absolute positional encoding assigns a representation to each individual position. Many relationships between tokens, however, are naturally described by the displacement between their positions. If two tokens occur at positions m and n, their signed relative displacement is

$$\Delta = m - n. \tag{2.55}$$

For example, the pairs $(m, n) = (10, 7)$ and $(m, n) = (100, 97)$ have different absolute positions but the same displacement, $\Delta = 3$. A relative positional method aims to make the interaction between the tokens depend on this relationship. One family of relative methods uses learnable representations indexed by relative displacement. Rotary Position Embedding (RoPE) achieves a related objective by rotating query and key representations according to their absolute positions so that their inner product depends on the relative displacement (**zhou2024rethinking**; **zhang2024foundmiddle**).

Two-dimensional rotations. The mathematical basis of RoPE is a rotation in a two-dimensional real vector space. For an angle ϕ, define

$$R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}. \tag{2.56}$$

For a vector $\mathbf{x} \in \mathbb{R}^2$, the rotated vector is $R(\phi)\mathbf{x}$. The matrix is orthogonal:

$$R(\phi)^\mathsf{T} R(\phi) = I, \qquad R(\phi)^\mathsf{T} = R(-\phi). \tag{2.57}$$

It follows that rotations preserve the Euclidean norm,

$$\|R(\phi)\mathbf{x}\|_2^2 = \mathbf{x}^\mathsf{T} R(\phi)^\mathsf{T} R(\phi)\mathbf{x} = \|\mathbf{x}\|_2^2. \tag{2.58}$$

The composition of two rotations adds their angles:

$$R(\alpha)R(\beta) = R(\alpha + \beta). \tag{2.59}$$

These identities are the essential algebraic properties used by RoPE. Equivalently, a two-dimensional vector can be represented as a complex number $z = x_1 + \mathrm{i}x_2$, and multiplication by $e^{\mathrm{i}\phi}$ performs the same rotation through Euler's identity.

Frequency-dependent rotations. Let the query and key dimension of one attention head be d_h, assumed even. RoPE groups the vector coordinates into $d_h/2$ pairs and assigns a different angular frequency to each pair. With the frequency convention used in the papers considered here,

$$\theta_j = 10000^{-2j/d_h}, \qquad j = 0, 1, \ldots, \frac{d_h}{2} - 1. \tag{2.60}$$

For position m, the complete rotation is a block-diagonal matrix,

$$R_m = \mathrm{diag}\left(R(m\theta_0), R(m\theta_1), \ldots, R(m\theta_{d_h/2-1})\right). \tag{2.61}$$

The transformed query and key vectors are

$$\widetilde{\mathbf{q}}_m = R_m \mathbf{q}_m, \qquad \widetilde{\mathbf{k}}_n = R_n \mathbf{k}_n. \tag{2.62}$$

The query-key inner product is then

$$\begin{aligned} \widetilde{\mathbf{q}}_m^{\mathsf{T}} \widetilde{\mathbf{k}}_n &= (R_m \mathbf{q}_m)^{\mathsf{T}} (R_n \mathbf{k}_n) && (2.63)\\ &= \mathbf{q}_m^{\mathsf{T}} R_m^{\mathsf{T}} R_n \mathbf{k}_n && (2.64)\\ &= \mathbf{q}_m^{\mathsf{T}} R_{n-m} \mathbf{k}_n. && (2.65) \end{aligned}$$

The last equality follows from the orthogonality and composition properties in equations (2.57) and (2.59). It establishes the central property of RoPE: although each query and key is transformed using its own absolute position, their attention interaction contains a rotation determined by the relative displacement $n - m$. The sign of the displacement depends on the convention used for the rotation matrix; the essential property is dependence on the difference between positions. The structure is summarized in figure 2.4.

The multiple frequencies in equation (2.60) cause different coordinate pairs to rotate at different rates. Higher-frequency components change phase more rapidly as position changes, while lower-frequency components change more gradually. RoPE therefore introduces positional information at multiple frequency scales directly into query-key interactions. It does not require a separate learned positional embedding table, because the transformations are determined by the position indices and the frequency schedule (**zhang2024foundmiddle**).

2.2.5 Long-Context Position Rescaling and Multi-Scale Positional Encoding

The ability to encode relative displacement does not guarantee that a language model will use all positions in a long input equally well. **zhang2024foundmiddle** study the "lost-in-the-middle" phenomenon: relevant information can be overlooked when it occurs in the

middle of a long context, even when the model can process the complete sequence. They identify two contributing factors. First, causal attention can favor initial tokens because these tokens are available to a larger number of subsequent queries. Second, the long-distance behavior associated with RoPE can reduce attention to distant tokens, including tokens that contain relevant information.

The latter observation should be interpreted as an empirical property of the resulting attention behavior, not as a theorem that every individual RoPE query-key score decreases monotonically with distance. The rotation matrices themselves preserve vector norms, and their inner-product contributions vary with relative phase. The paper's motivation is that the combined behavior of RoPE-based attention in the evaluated models can contribute to reduced use of distant context.

Uniform positional rescaling. Since RoPE uses position indices to determine rotation angles, the effective positional distances can be changed by rescaling the indices. For a positive scaling factor r, replace position m by

$$m' = \frac{m}{r}. \tag{2.66}$$

For two positions m and n, the rescaled displacement becomes

$$\Delta' = \frac{m}{r} - \frac{n}{r} = \frac{m-n}{r} = \frac{\Delta}{r}. \tag{2.67}$$

When $r > 1$, the effective positional distance is compressed. When $0 < r < 1$, it is expanded, and $r = 1$ recovers the original RoPE positions. For example, two tokens separated by $3{,}000$ positions have an effective separation of

$$\Delta' = \frac{3000}{1.5} = 2000 \tag{2.68}$$

when $r = 1.5$. The sequence itself is not shortened and no tokens are removed; only the positional indices used by the rotary transformation are remapped. This can reduce the phase difference between distant positions and potentially alleviate the long-distance decay observed by the authors.

Uniform rescaling has a trade-off. A factor that is too large can alter the positional patterns learned during pre-training and can change which regions of the context the model favors. Fractional rescaled positions may also differ from the integer position indices encountered during pre-training, creating a positional out-of-distribution concern. In the experiments reported by Zhang et al., small scaling factors such as 0.5 favor more recent context, whereas larger factors such as 2.5 can favor the beginning of the context. A factor near 1.5 improves average performance in their initial rescaling experiments, but a single factor does not account for the different positional behavior of individual attention heads.

Position-awareness across attention heads. The motivation for Multi-Scale Positional Encoding (Ms-PoE) is the observation that attention heads differ in their ability to follow relevant information when its position changes. Some heads shift their attention toward the relevant content as that content moves through the input. Other heads remain more strongly concentrated on the beginning and end regions. Zhang et al. use

the term *position-aware* for heads that can capture relevant information across these positional changes.

For a layer k, let the attention distribution of head h for the most recent query be

$$\mathbf{A}_{k,h} = [A_{k,h,1}, A_{k,h,2}, \ldots, A_{k,h,L}], \qquad \sum_{t=1}^{L} A_{k,h,t} = 1. \tag{2.69}$$

Its mean component is $1/L$. The Position-Awareness Score is defined as

$$S^{P}_{k,h} = \frac{1}{L} \sum_{t=1}^{L} \mathbb{I}\left(A_{k,h,t} \geq \alpha \frac{1}{L} \sum_{j=1}^{L} A_{k,h,j} \right), \tag{2.70}$$

where $\mathbb{I}(\cdot)$ is the indicator function and α is a threshold hyperparameter. The experiments use $\alpha = 3$. Because the attention weights sum to one, the threshold in this case is $3/L$. The score is the fraction of positions whose attention weights are at least three times the mean attention weight. A larger value indicates that a larger fraction of positions receive attention above this threshold. This score is an operational measure introduced by the authors; it is not itself a direct measure of semantic understanding or retrieval accuracy.

For each layer, heads are sorted in descending order of their Position-Awareness Scores. The most position-aware head receives a rescaling factor near one, while heads with lower scores receive larger factors. The authors use a linearly spaced set of factors from 1.2 to 1.8. For the head at sorted rank i, the assigned factor is

$$r_i = 1.2 + \frac{(i-1)(1.8-1.2)}{n_h - 1}, \qquad i = 1, \ldots, n_h, \tag{2.71}$$

where n_h is the number of attention heads in the layer. Thus, the first head in the descending positional-awareness ordering receives $r_1 = 1.2$, and the last receives $r_{n_h} = 1.8$. The factors are assigned separately within each layer. The resulting head-specific positional transformation is

$$\widetilde{\mathbf{q}}_{k,h,m} = R\left(\frac{m}{r_{k,h}}\right) \mathbf{q}_{k,h,m}, \qquad \widetilde{\mathbf{k}}_{k,h,n} = R\left(\frac{n}{r_{k,h}}\right) \mathbf{k}_{k,h,n}, \tag{2.72}$$

where $R(\cdot)$ denotes the frequency-dependent block rotation defined earlier. The relative displacement used by head h in layer k is consequently $(m-n)/r_{k,h}$. Different heads therefore operate with different effective positional scales. The arrangement is illustrated in figure 2.5.

Inference procedure and computational considerations. Ms-PoE is applied to a pretrained model that already uses RoPE. During prefilling, the model obtains the attention distribution of the most recent query for every head in every layer. It computes the Position-Awareness Scores, sorts the heads within each layer, and assigns the linearly spaced rescaling factors. These factors remain fixed during the subsequent decoding stage. The procedure does not require additional fine-tuning or additional trainable positional parameters (**zhang2024foundmiddle**).

Algorithm 1: Head-wise positional rescaling in Ms-PoE

Input: Pre-trained RoPE-based language model; input sequence; number of layers n_l; number of heads n_h; threshold α; scaling interval $[1.2, 1.8]$.

Output: Generated sequence using head-specific positional rescaling.

for $k = 1$ **to** n_l **do**
  Obtain the most recent query and the keys for layer k;
  for $h = 1$ **to** n_h **do**
    Obtain the attention distribution $\mathbf{A}_{k,h}$;
    Compute $S^P_{k,h}$ using equation (2.70);
  Sort the heads by $S^P_{k,h}$ in descending order;
  Assign the factors r_i using equation (2.71);
  Associate each assigned factor with its sorted head;
Replace the original RoPE position indices by the corresponding head-specific rescaled indices during decoding;
Generate tokens autoregressively using the fixed factors;

The attention vectors used for the score are already part of the prefilling attention computation. Given these vectors, evaluating the threshold indicator for all heads and positions requires $O(n_l n_h L)$ comparisons. Sorting the n_h heads in each layer requires $O(n_l n_h \log n_h)$ comparisons. Storing the scores and assigned factors requires $O(n_l n_h)$ scalar values. These costs describe the additional score-reduction and ordering operations implied by the procedure; the method does not introduce a new trainable parameter matrix or a new attention layer. Once the factors are fixed, the decoding stage uses the modified RoPE transformations for the existing heads.

Experimental evaluation. Zhang et al. evaluate Ms-PoE on long-context generation and retrieval tasks using Llama-2, StableBeluga, and Vicuna models. On the ZeroSCROLLS benchmark, they report average improvements over the corresponding baselines for the evaluated models. The reported average gain is 1.6 points for Llama-2-7B-Chat, 0.9 for Llama-2-13B-Chat, 2.6 for StableBeluga-7B, 0.6 for StableBeluga-13B, 3.8 for Vicuna-7B, and 3.3 for Vicuna-7B-16K. The results indicate improvements across the reported model sizes and context configurations, although the magnitude varies by model and task (**zhang2024foundmiddle**).

The multi-document question-answering and key-value retrieval experiments vary the location of the relevant document or key-value pair. For Vicuna-7B, the average multi-document question-answering accuracy increases from 61.12% for the baseline to 65.04% with Ms-PoE. For key-value retrieval, the corresponding average increases from 36.24% to 79.96%. The authors average results over 500 samples and use the difference between the best and worst accuracy across relevant-information positions as a measure of positional sensitivity:

$$\text{GapAccuracy} = \max_p \text{Acc}(p) - \min_p \text{Acc}(p), \tag{2.73}$$

where $\text{Acc}(p)$ is the accuracy when the relevant information is placed at position p. A smaller gap indicates less variation in accuracy as the location of relevant information changes. The results show that Ms-PoE improves average retrieval performance and reduces the positional gap in the evaluated settings (**zhang2024foundmiddle**).

The ablation experiments also examine the way scaling factors are assigned. In the reported multi-document question-answering experiment, the position-awareness ordering achieves an average accuracy of 65.3%, compared with 61.7% for random assignment, 57.8% for sequential assignment, and 62.3% for attention-entropy-based assignment. The baseline average is 62.1%. These results support the authors' claim that head-wise positional-awareness information provides a more effective assignment criterion than the tested alternatives. They do not establish that the same ordering criterion is optimal for every architecture or task.

2.2.6 Limitations and Connection to the Next Architectural Components

Positional encoding determines how a Transformer represents token locations and how positional information enters attention interactions. The methods considered here make different design choices. Sinusoidal encoding imposes a fixed multiscale trigonometric structure; learnable absolute encoding estimates a positional table during training; addition and concatenation determine how content and position are combined; RoPE uses position-dependent rotations whose query-key interaction depends on relative displacement; and Ms-PoE modifies RoPE at inference by assigning different rescaling factors to different attention heads.

Each construction has limitations. A learned absolute positional table is tied to the positions represented by its parameters. Sinusoidal encoding supplies a fixed frequency structure, but the model must still learn to use that structure through its projections. Concatenation changes the parameterization of content-position interactions, but the experimental comparison reported by Zhou et al. concerns vision architectures and cannot by itself establish an advantage for language models. RoPE provides a relative-position interaction through rotations, but its behavior over long distances can still be associated with reduced utilization of distant context. Ms-PoE addresses this issue in the evaluated models, but its head-wise score is a threshold-based proxy, its factors are selected from a specified interval, and rescaling can create positional patterns that differ from those seen during pre-training.

The central architectural conclusion is that positional encoding is not merely an auxiliary vector added to token embeddings. It determines how sequence order and relative displacement influence the attention computation, and therefore affects how a language model can represent and use information distributed across a sequence. With token content and positional relationships now specified as inputs to the attention mechanism, the next architectural question is how each token representation is further transformed after attention. This leads to the feed-forward component of the Transformer, which applies position-wise nonlinear transformations to the representations constructed by attention.

2.3 Feed-Forward Networks

Positional encoding supplies token representations with information about their positions in a sequence. Position information, however, does not by itself specify how the features represented by each token are transformed into increasingly useful internal representations. A Transformer therefore requires computational components that operate on the feature dimensions of token representations in addition to components that es-

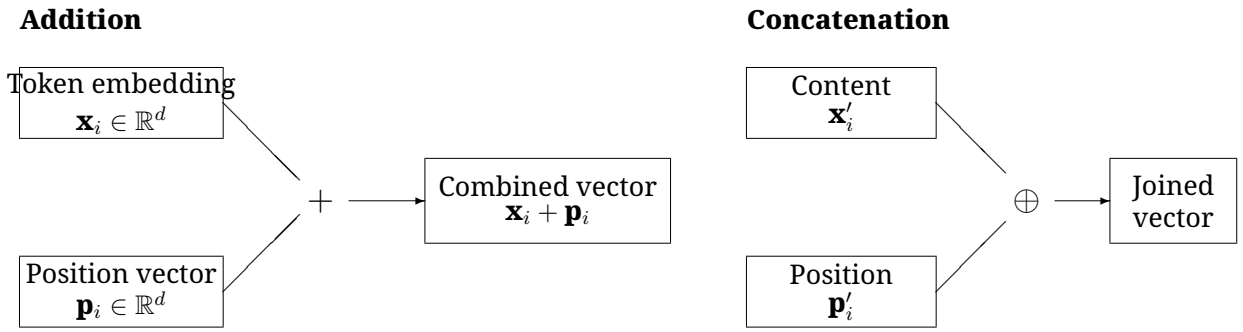
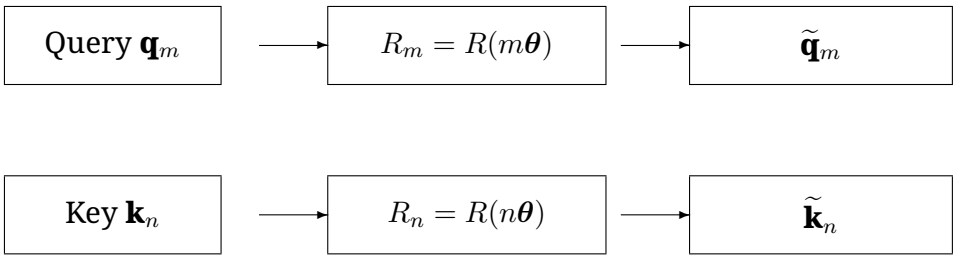


Figure 2.3: Two ways of incorporating positional information into token representations. Addition superimposes full-dimensional vectors, whereas concatenation joins half-dimensional content and positional components while retaining the same total dimension. Adapted conceptually from the comparison in **zhou2024rethinking**.



$$\widetilde{\mathbf{q}}_m^{\mathsf{T}} \widetilde{\mathbf{k}}_n = \mathbf{q}_m^{\mathsf{T}} R_{n-m} \mathbf{k}_n$$

Figure 2.4: Rotary Position Embedding applies position-dependent rotations to queries and keys. Their inner product contains a rotation determined by the relative displacement between positions. Schematic based on **zhang2024foundmiddle**.

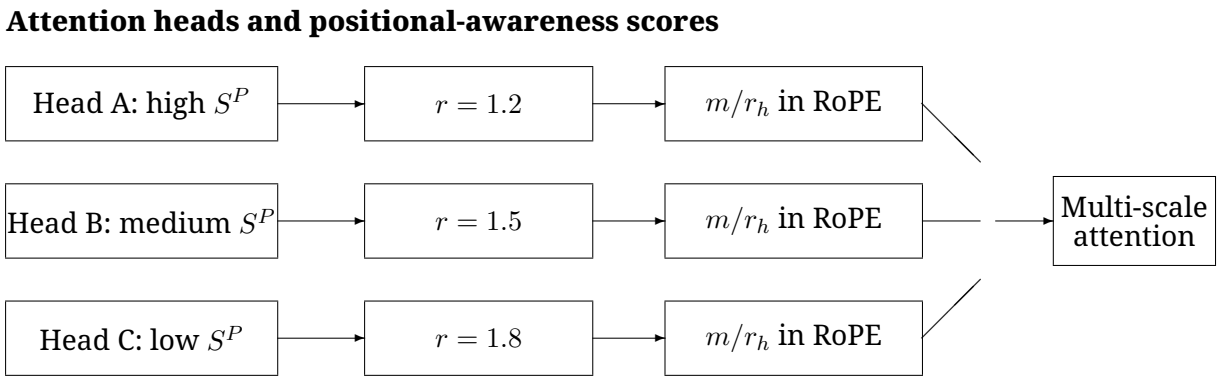


Figure 2.5: Conceptual flow of Ms-PoE. Heads are ranked by their position-awareness scores and receive different rescaling factors; higher-scoring heads are changed less, while lower-scoring heads receive stronger positional compression. The values shown are illustrative points in the interval used by **zhang2024foundmiddle**.

tablish relationships between different token positions. The feed-forward network (FFN) provides this feature-transformation mechanism. In a conventional Transformer block, the FFN is applied independently to each token representation, using the same learned parameters at every sequence position (**dai2024deepseekmoe**; **cai2025survey**).

The distinction between position-wise processing and sequence-wise interaction is fundamental. An FFN transforms the feature vector associated with one token without directly combining it with the feature vectors of other tokens. The parameters are shared across positions, so the same nonlinear mapping is applied throughout the sequence. This arrangement complements the self-attention sublayer, which supplies interactions between token positions. The present section develops the mathematical structure of the FFN, examines its parameter and computational requirements, and then follows the architectural transition from dense FFNs to sparsely activated mixtures of FFN experts.

2.3.1 Position-Wise Feed-Forward Networks

Let a sequence contain T tokens, and let d_{model} denote the dimensionality of each token representation. The representation of token t is a vector $\mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}$. A sequence can be represented by a matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T},$$

where each column corresponds to one token. This column-wise convention will be used throughout the mathematical development.

A standard two-projection FFN first maps the input representation into an intermediate feature space, applies an element-wise nonlinear activation, and then maps the result back into the model hidden space. For one token, the computation is

$$\mathbf{a}_t = \mathbf{W}_1 \mathbf{x}_t + \mathbf{b}_1, \tag{2.74}$$

$$\widetilde{\mathbf{a}}_t = \phi(\mathbf{a}_t), \tag{2.75}$$

$$\mathbf{y}_t = \mathbf{W}_2 \widetilde{\mathbf{a}}_t + \mathbf{b}_2. \tag{2.76}$$

Here, d_{ff} is the intermediate hidden dimension, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the input-projection matrix, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$ is its bias vector, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ is the output-projection matrix, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$ is the output bias. The activation function ϕ acts element-wise on its vector argument. The dimensions therefore satisfy

$$\mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}, \quad \mathbf{a}_t, \widetilde{\mathbf{a}}_t \in \mathbb{R}^{d_{\text{ff}}}, \quad \mathbf{y}_t \in \mathbb{R}^{d_{\text{model}}}.$$

For a bias-free formulation, the bias vectors are omitted. The essential structure is the same: an input projection, an element-wise nonlinearity, and an output projection. The conventional FFN is consequently a two-layer multilayer perceptron whose input and output dimensions agree, while its intermediate dimension may be wider. The description of standard FFN experts as linear–activation–linear networks is also used in the MoE literature (**dai2024deepseekmoe**; **cai2025survey**).

The complete sequence is processed by applying the same matrices and bias vectors to every column:

$$\mathbf{A} = \mathbf{W}_1 \mathbf{X} + \mathbf{b}_1 \mathbf{1}_T^\top, \tag{2.77}$$

$$\widetilde{\mathbf{A}} = \phi(\mathbf{A}), \tag{2.78}$$

$$\mathbf{Y} = \mathbf{W}_2 \widetilde{\mathbf{A}} + \mathbf{b}_2 \mathbf{1}_T^\top, \tag{2.79}$$

where $\mathbf{1}_T \in \mathbb{R}^T$ is the all-ones vector. Thus, $\mathbf{A}, \widetilde{\mathbf{A}} \in \mathbb{R}^{d_{\text{ff}} \times T}$ and $\mathbf{Y} \in \mathbb{R}^{d_{\text{model}} \times T}$. The matrix multiplication acts on the feature dimension of each column; there is no multiplication between different token columns. Therefore, for every t, the output $\mathbf{y}_t$ depends on $\mathbf{x}_t$ and the shared FFN parameters, but not directly on $\mathbf{x}_s$ for $s \neq t$.

The nonlinearity is necessary for the composition to represent a nonlinear mapping. If ϕ were the identity function and the biases were zero, then

$$\mathbf{y}_t = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x}_t.$$

The product $\mathbf{W}_2 \mathbf{W}_1$ is itself a matrix, so the two projections would collapse to one linear transformation. An intervening nonlinear activation prevents this reduction in general and allows the FFN to implement a nonlinear feature transformation. The specific mathematical properties of the activation function are treated in the subsequent section on activation functions.

2.3.2 Parameterization and Computational Complexity

The dimensional structure of an FFN determines its parameter count. Ignoring bias parameters, the input-projection matrix contains $d_{\text{ff}} d_{\text{model}}$ entries, and the output-

projection matrix contains $d_{\text{model}}d_{\text{ff}}$ entries. The total number of weight parameters is therefore

$$P_{\text{FFN}} = d_{\text{ff}}d_{\text{model}} + d_{\text{model}}d_{\text{ff}} \tag{2.80}$$
$$= 2d_{\text{model}}d_{\text{ff}}. \tag{2.81}$$

When both bias vectors are included, the total becomes

$$P_{\text{FFN,bias}} = 2d_{\text{model}}d_{\text{ff}} + d_{\text{ff}} + d_{\text{model}}. \tag{2.82}$$

It is useful to express the intermediate dimension relative to the model dimension. Define the expansion ratio

$$r = \frac{d_{\text{ff}}}{d_{\text{model}}}, \qquad d_{\text{ff}} = r\, d_{\text{model}}. \tag{2.83}$$

Substitution into equation (2.81) gives

$$P_{\text{FFN}} = 2r\, d_{\text{model}}^2. \tag{2.84}$$

For a fixed expansion ratio, the number of FFN weight parameters therefore grows quadratically with the model hidden dimension. The intermediate dimension is an architectural choice rather than a universal constant; its value depends on the model configuration.

The dominant arithmetic cost is incurred by the two matrix multiplications. Let M denote the total number of token vectors processed by the FFN in a batch. The first projection requires $Md_{\text{model}}d_{\text{ff}}$ multiply–accumulate operations (MACs), and the second projection requires the same number. Thus,

$$C_{\text{FFN}} = 2Md_{\text{model}}d_{\text{ff}} \quad \text{MACs}. \tag{2.85}$$

When a multiplication and an addition are counted as separate floating-point operations, the corresponding leading-order arithmetic count is

$$F_{\text{FFN}} \approx 4Md_{\text{model}}d_{\text{ff}} \quad \text{FLOPs}. \tag{2.86}$$

This estimate counts the two dominant projections. Bias additions and element-wise activation operations add work of order $O(Md_{\text{ff}})$, which is lower order than the matrix multiplications when the model dimensions are large. The estimate also uses the conventional dense-matrix arithmetic count; actual latency depends on hardware, precision, memory movement, and kernel efficiency.

The distinction between parameter count and arithmetic cost is important. The matrices are shared across all token positions, so changing the sequence length does not change the number of FFN parameters. It does, however, change the number of times those parameters are used and therefore changes the computation approximately linearly in M. Increasing d_{model} or d_{ff}, in contrast, changes both parameter count and per-token computation.

2.3.3 Gated Feed-Forward Networks and Their Internal Units

The two-projection FFN is a useful basic model, but modern language-model FFNs can introduce an additional projection that controls the intermediate features. Lo et al. examine the three-projection expert structure used in LLaMA-style MoE models and write its computation as (**lo2025closer**):

$$E(\mathbf{x}) = \mathbf{W}_{\text{down}} \left[\left(\mathbf{W}_{\text{up}} \mathbf{x} \right) \odot \sigma \left(\mathbf{W}_{\text{act}} \mathbf{x} \right) \right]. \tag{2.87}$$

Here, $\mathbf{x} \in \mathbb{R}^{d_{\text{hid}}}$, $\mathbf{W}_{\text{up}}, \mathbf{W}_{\text{act}} \in \mathbb{R}^{d_{\text{mid}} \times d_{\text{hid}}}$, and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{mid}}}$. The symbol $\odot$ denotes element-wise multiplication, and σ is an element-wise activation function. Both the input and output of the expert belong to $\mathbb{R}^{d_{\text{hid}}}$, while the two intermediate projections produce vectors in $\mathbb{R}^{d_{\text{mid}}}$.

The up projection produces

$$\mathbf{a} = \mathbf{W}_{\text{up}} \mathbf{x}, \tag{2.88}$$

whereas the activation projection produces

$$\mathbf{g} = \sigma \left(\mathbf{W}_{\text{act}} \mathbf{x} \right). \tag{2.89}$$

Their element-wise product is

$$\mathbf{z} = \mathbf{a} \odot \mathbf{g}, \qquad z_i = a_i g_i, \quad i = 1, \ldots, d_{\text{mid}}. \tag{2.90}$$

The down projection then maps this modulated intermediate representation back to the hidden dimension:

$$E(\mathbf{x}) = \mathbf{W}_{\text{down}} \mathbf{z}. \tag{2.91}$$

The additional projection is not merely a different choice of activation function. It supplies a second learned transformation of the input, and its output controls the intermediate features through element-wise multiplication.

Ignoring biases, the three matrices contain

$$P_{\text{gated}} = d_{\text{mid}} d_{\text{hid}} + d_{\text{mid}} d_{\text{hid}} + d_{\text{hid}} d_{\text{mid}} \tag{2.92}$$

$$= 3 d_{\text{hid}} d_{\text{mid}} \tag{2.93}$$

parameters. This is 50% more projection parameters than a two-projection FFN with the same input, output, and intermediate dimensions. For M tokens, the three projections require approximately $3 M d_{\text{hid}} d_{\text{mid}}$ MACs, or $6 M d_{\text{hid}} d_{\text{mid}}$ FLOPs when each multiply–accumulate is counted as two FLOPs, in addition to element-wise operations.

The dimensions reported by Lo et al. for the Mistral 7B configuration include $d_{\text{hid}} = 4096$ and $d_{\text{mid}} = 14336$. Using these dimensions in the three-projection expression gives

$$P_{\text{gated}} = 3(4096)(14336) \tag{2.94}$$

$$= 176\,160\,768. \tag{2.95}$$

This is the weight count for the three projection matrices under the bias-free formulation. It is not a count of every parameter in the Transformer block or the complete language model (**lo2025closer**).

The three-projection formulation also permits the contribution of each intermediate unit to be examined explicitly. Let $\mathbf{W}_{\text{up}}[i, :]$ and $\mathbf{W}_{\text{act}}[i, :]$ denote the i-th rows of the up

and activation matrices, and let $\mathbf{W}_{\text{down}}[:, i]$ denote the i-th column of the down matrix. The contribution of unit i is determined by

$$z_i = \left(\mathbf{W}_{\text{up}}[i, :]\mathbf{x}\right) \sigma\left(\mathbf{W}_{\text{act}}[i, :]\mathbf{x}\right). \tag{2.96}$$

The complete expert output can consequently be decomposed as

$$E(\mathbf{x}) = \sum_{i=1}^{d_{\text{mid}}} \mathbf{W}_{\text{down}}[:, i] \left[\left(\mathbf{W}_{\text{up}}[i, :]\mathbf{x}\right) \sigma\left(\mathbf{W}_{\text{act}}[i, :]\mathbf{x}\right)\right]. \tag{2.97}$$

Each intermediate unit therefore combines two input-side projections with an output-side vector. The input projections determine the input-dependent scalar activation of that unit, and the corresponding column of the down matrix determines the direction of its contribution in the output space.

Lo et al. use this decomposition to motivate the interpretation of FFN neurons as fine-grained computational experts. This is a functional interpretation of the internal units, not a claim that each neuron is an independent expert network with its own router. The activation projection controls intermediate neurons within one FFN, whereas an MoE router selects among complete expert modules. These are distinct levels of computation (**lo2025closer**).

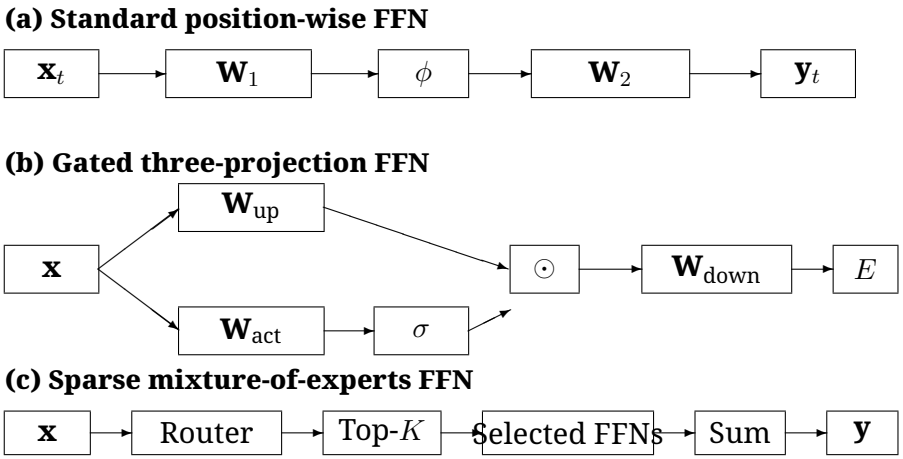


Figure 2.6: Schematic comparison of a standard position-wise FFN, a three-projection gated FFN, and a sparsely routed MoE layer. In the standard FFN, the input is projected, transformed element-wise, and projected back to the model dimension. In the gated FFN, parallel up and activation projections are combined element-wise before the down projection. In the sparse MoE layer, a router selects a subset of FFN experts whose weighted outputs are aggregated.

2.3.4 Feed-Forward Networks Within Transformer Blocks

An FFN is a component of a larger Transformer block rather than an independent sequence-processing stage. In the formulation used by Dai et al., the self-attention sublayer first produces a representation with a residual connection, after which the

FFN transforms each token representation and adds another residual connection (**dai2024deepseekmoe**). Let $\mathbf{h}_{1:T}^{l-1}$ denote the token representations entering block l, and let $\mathbf{u}_{1:T}^{l}$ denote the representations after its self-attention sublayer. The simplified computation is

$$\mathbf{u}_{1:T}^{l} = \text{SelfAtt}\left(\mathbf{h}_{1:T}^{l-1}\right) + \mathbf{h}_{1:T}^{l-1}. \tag{2.98}$$

The FFN then operates on each token separately:

$$\mathbf{h}_{t}^{l} = \text{FFN}\left(\mathbf{u}_{t}^{l}\right) + \mathbf{u}_{t}^{l}. \tag{2.99}$$

The residual addition requires the FFN output to have the same dimension as its input. This is why the second FFN projection returns the intermediate representation to d_{model} dimensions.

A residual connection adds the input of a sublayer to the transformation computed by that sublayer. It therefore preserves a direct path for the incoming representation while allowing the learned transformation to contribute an update. The equations above follow the simplified formulation in Dai et al.; that paper omits layer-normalization operations from these equations for brevity. The equations should therefore be read as a description of the FFN and residual structure, not as a complete specification of every normalization convention used by all Transformer implementations (**dai2024deepseekmoe**).

The two sublayers have complementary roles. Self-attention explicitly combines information across token positions, whereas the position-wise FFN applies the same learned nonlinear feature transformation to each token. The FFN does not itself create cross-token interactions; it transforms the representation that is available at its input.

2.3.5 From Dense FFNs to Mixture-of-Experts Layers

The parameter count of an FFN can be substantial because both projection matrices scale with the model and intermediate dimensions. Cai et al. report that, in the 540-billion-parameter PaLM configuration discussed in their survey, approximately 90% of the parameters are located in FFN layers. They also summarize an empirical analysis reporting lower active expert engagement in FFN layers than in self-attention layers for the particular model and task examined. These observations motivate the investigation of conditional computation in FFN modules, but the reported proportions are specific to the cited experiments and should not be treated as universal constants for Transformer models (**cai2025survey**).

A mixture-of-experts (MoE) layer replaces one FFN with a collection of N expert networks,

$$\mathcal{E} = \{E_1, E_2, \ldots, E_N\},$$

where each expert is itself an FFN. A gating network, also called a router, assigns scores to the experts for each input token. In a linear-router formulation, the router logits for input $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ are

$$\mathbf{q}(\mathbf{x}) = \mathbf{W}_r \mathbf{x}, \qquad \mathbf{W}_r \in \mathbb{R}^{N \times d_{\text{model}}}. \tag{2.100}$$

The logits are transformed into routing probabilities by the softmax:

$$p_i(\mathbf{x}) = \frac{\exp(q_i(\mathbf{x}))}{\sum_{j=1}^{N} \exp(q_j(\mathbf{x}))}, \qquad \sum_{i=1}^{N} p_i(\mathbf{x}) = 1. \tag{2.101}$$

The probabilities are nonnegative and sum to one, so they define a distribution over the available experts.

In a dense MoE layer, all experts are evaluated and their outputs are combined using their routing probabilities:

$$\mathrm{MoE}_{\mathrm{dense}}(\mathbf{x}) = \sum_{i=1}^{N} p_i(\mathbf{x}) E_i(\mathbf{x}). \tag{2.102}$$

In a sparse MoE layer, only the K experts with the largest routing scores are evaluated. Let $\mathcal{T}_K(\mathbf{x})$ denote the set of their indices. The sparse gate is

$$g_i(\mathbf{x}) = \begin{cases} p_i(\mathbf{x}), & i \in \mathcal{T}_K(\mathbf{x}), \\ 0, & i \notin \mathcal{T}_K(\mathbf{x}). \end{cases} \tag{2.103}$$

The sparse MoE output is

$$\mathrm{MoE}_{\mathrm{sparse}}(\mathbf{x}) = \sum_{i=1}^{N} g_i(\mathbf{x}) E_i(\mathbf{x}) = \sum_{i \in \mathcal{T}_K(\mathbf{x})} p_i(\mathbf{x}) E_i(\mathbf{x}). \tag{2.104}$$

The distinction is computational: in the sparse formulation, experts whose gate values are zero need not perform their FFN computation for that token. The linear-router formulation above follows the routing description in Fedus et al.; Dai et al. also describe token-to-expert affinity scores based on the similarity between a token representation and an expert centroid (**fedus2022switch**; **dai2024deepseekmoe**).

If each expert contains P_E parameters and all experts have the same size, the total expert parameter count is

$$P_{\mathrm{experts,total}} = N P_E. \tag{2.105}$$

When K experts are active for a token, the number of active expert parameters is

$$P_{\mathrm{experts,active}} = K P_E. \tag{2.106}$$

The ratio is therefore

$$\frac{P_{\mathrm{experts,active}}}{P_{\mathrm{experts,total}}} = \frac{K}{N}. \tag{2.107}$$

These expressions concern expert parameters and exclude router parameters and any other shared components. They show how a sparse MoE layer can expose a large total pool of FFN parameters while activating only a subset for an individual token. The total model capacity and the expert computation per token are consequently different quantities.

2.3.6 Switch Routing, Expert Capacity, and Load Balancing

Fedus et al. introduce the Switch Transformer as a simplified sparse MoE design in which each token is routed to one expert rather than to multiple experts. The selected expert is

$$i^* = \arg\max_{1 \le i \le N} p_i(\mathbf{x}), \tag{2.108}$$

and the output of the Switch FFN is

$$\text{SwitchFFN}(\mathbf{x}) = p_{i^*}(\mathbf{x}) E_{i^*}(\mathbf{x}). \tag{2.109}$$

The expert output is scaled by the selected router probability. Top-1 routing reduces the number of expert computations per token and simplifies token dispatch relative to a routing rule that activates multiple experts. Fedus et al. report that this simplification can preserve model quality in their experiments while reducing routing and communication requirements (**fedus2022switch**).

Sparse routing creates a finite-capacity allocation problem. Let M be the number of tokens presented to one MoE layer in a batch. Under uniform allocation across N experts, the expected number of tokens per expert is M/N. Switch Transformer defines expert capacity as

$$C = \left(\frac{M}{N} \right) c_{\text{factor}}, \tag{2.110}$$

where c_{factor} is the capacity factor. A factor greater than one provides buffer for nonuniform routing. In an implementation, the resulting capacity must be represented by an admissible integer tensor dimension.

For example, consider $M = 1024$ tokens and $N = 16$ experts. The uniform allocation is $1024/16 = 64$ tokens per expert. With $c_{\text{factor}} = 1.5$, the capacity is

$$C = 64 \times 1.5 = 96.$$

An expert can therefore accept up to 96 tokens in this example. If more tokens are routed to it, the excess tokens are dropped from that expert computation in the Switch implementation and their representations are passed to the next layer through the residual connection. A larger capacity factor reduces the risk of overflow, but it also allocates more buffer and can increase memory and computation spent on unused capacity. Capacity is therefore a trade-off between token overflow and resource utilization (**fedus2022switch**).

The router may not distribute tokens uniformly. To encourage a more balanced allocation, Switch Transformer adds an auxiliary loss to the main training objective. For a batch $\mathcal{B}$ containing M tokens, define the fraction of tokens dispatched to expert i as

$$f_i = \frac{1}{M} \sum_{\mathbf{x} \in \mathcal{B}} \mathbb{1} \left[\arg\max_j p_j(\mathbf{x}) = i \right], \tag{2.111}$$

where $\mathbb{1}[\cdot]$ is the indicator function. Define the average router probability assigned to expert i as

$$P_i = \frac{1}{M} \sum_{\mathbf{x} \in \mathcal{B}} p_i(\mathbf{x}). \tag{2.112}$$

The auxiliary load-balancing loss is

$$\mathcal{L}_{\text{aux}} = \alpha N \sum_{i=1}^{N} f_i P_i, \tag{2.113}$$

where α controls the contribution of this term to the training objective. The vector f records the realized dispatch frequencies, whereas P records the average probability

mass assigned by the router. These quantities are not interchangeable: one describes discrete assignments and the other describes the router's probability outputs.

Uniform allocation corresponds to $f_i = P_i = 1/N$ for every expert. Under this condition,

$$\mathcal{L}_{\text{aux}} = \alpha N \sum_{i=1}^{N} \frac{1}{N^2} = \alpha.$$

The objective is minimized at the uniform allocation. Although the dispatch indicator in f_i is discrete, the probability term P_i depends on the softmax router outputs and supplies a differentiable training signal to the router. The auxiliary loss is added to the model's main loss; it does not replace the language-modeling objective (**fedus2022switch**).

2.3.7 Fine-Grained Segmentation and Shared Experts

A conventional sparse MoE layer may contain a limited number of relatively large FFN experts. Dai et al. identify two potential limitations of this organization. Knowledge hybridity occurs when one expert must process tokens associated with diverse types of knowledge. Knowledge redundancy occurs when different experts independently acquire overlapping information that is useful across multiple contexts. DeepSeekMoE addresses these problems through fine-grained expert segmentation and shared expert isolation (**dai2024deepseekmoe**).

In fine-grained expert segmentation, each original expert is divided into m smaller experts by reducing its intermediate dimension by a factor of m. If the original architecture has N experts, each with intermediate dimension d_{ff}, the fine-grained architecture has

$$N_{\text{fine}} = mN, \qquad d_{\text{ff,fine}} = \frac{d_{\text{ff}}}{m}. \tag{2.114}$$

To preserve the dominant expert computation per token, the number of activated experts is increased from K to mK:

$$K_{\text{fine}} = mK. \tag{2.115}$$

These relations assume that the intermediate dimension can be divided as specified and that the expert projection costs scale linearly with that dimension.

If an original expert contains P_E projection parameters, each fine-grained expert contains approximately P_E/m such parameters. The total expert parameter count is therefore preserved:

$$P_{\text{fine,total}} = mN \left(\frac{P_E}{m} \right) \tag{2.116}$$

$$= NP_E. \tag{2.117}$$

Likewise, if the dominant computation of one original expert is C_E, then the active expert computation is

$$C_{\text{fine,active}} = mK \left(\frac{C_E}{m} \right) \tag{2.118}$$

$$= KC_E. \tag{2.119}$$

Thus, the segmentation increases the number of available experts and the number selected for each token while preserving the leading-order expert parameter count and active expert computation.

The principal change is the number of possible expert combinations. If K experts are selected from N experts, the number of unordered combinations is

$$\binom{N}{K} = \frac{N!}{K!(N-K)!}. \tag{2.120}$$

After segmentation, the corresponding count is

$$\binom{mN}{mK}. \tag{2.121}$$

Dai et al. give the example $N = 16$ and $K = 2$, for which the conventional architecture permits

$$\binom{16}{2} = 120$$

possible combinations. With $m = 4$, the fine-grained architecture contains 64 experts and activates eight, giving

$$\binom{64}{8} = 4\,426\,165\,368$$

possible combinations. This is a count of possible subsets, not a claim that all combinations are equally likely or are observed during training. The larger combination space gives the router more ways to compose specialized expert subsets (**dai2024deepseekmoe**).

Fine-grained segmentation does not, by itself, eliminate redundancy between experts. DeepSeekMoE therefore introduces shared expert isolation. A designated set of K_s shared experts is activated for every token, while the remaining experts are routed conditionally. The number of routed experts is $mN - K_s$, and the number of routed experts selected for each token is $mK - K_s$. A simplified representation of the layer output is

$$\begin{aligned}\mathbf{h}_t^l = &\sum_{j=1}^{K_s} E_j^{\text{shared}}\left(\mathbf{u}_t^l\right) \\ &+ \sum_{i=1}^{mN-K_s} g_{i,t} E_i^{\text{routed}}\left(\mathbf{u}_t^l\right) + \mathbf{u}_t^l.\end{aligned} \tag{2.122}$$

The shared experts provide a common computational pathway for information that is useful across contexts. The routed experts can then specialize more strongly in input-dependent information without each having to reproduce all common knowledge.

For the routing formulation described by Dai et al., the affinity between token t and expert i in layer l is computed from the token representation and the expert centroid $\mathbf{e}_i^l$:

$$s_{i,t} = \text{Softmax}_i\left((\mathbf{u}_t^l)^\top \mathbf{e}_i^l\right). \tag{2.123}$$

The sparse gate retains the mK largest affinities in the fine-grained formulation, with shared experts handled separately. The centroid-affinity expression is a particular routing parameterization and should not be conflated with the linear router used in the generic Switch formulation.

The DeepSeekMoE paper distinguishes balancing at the expert level from balancing at the device level. Let

$$N' = mN - K_s, \qquad K' = mK - K_s$$

denote the number of routed experts and the number of routed experts selected for each token, respectively. For a batch of T tokens, the normalized dispatch fraction for routed expert i is

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}\left(\text{token } t \text{ selects routed expert } i\right), \tag{2.124}$$

and the average affinity assigned to that expert is

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}. \tag{2.125}$$

The expert-level balancing objective is

$$\mathcal{L}_{\text{ExpBal}} = \frac{\alpha_1}{N'} \sum_{i=1}^{N'} f_i P_i, \tag{2.126}$$

where α_1 controls the strength of expert-level balancing. The normalization in f_i makes its value equal to one under uniform routed-expert allocation.

Balancing individual experts does not guarantee balanced accelerator workloads. Suppose the routed experts are partitioned into D groups, with group $\mathcal{G}_d$ deployed on device d. The normalized dispatch fraction and aggregate affinity for device group d are

$$f'_d = \frac{1}{|\mathcal{G}_d|} \sum_{j \in \mathcal{G}_d} f_j, \tag{2.127}$$

$$P'_d = \sum_{j \in \mathcal{G}_d} P_j. \tag{2.128}$$

The corresponding device-level balancing objective is

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{d=1}^{D} f'_d P'_d, \tag{2.129}$$

where α_2 is the device-level balance factor. Dai et al. describe using a relatively small expert-level balance factor to reduce routing collapse while assigning a larger weight to device-level balancing, because overly strict expert-level balancing can compromise model performance. These two objectives address different bottlenecks: one concerns token allocation among experts, and the other concerns the aggregate computational load assigned to devices (**dai2024deepseekmoe**).

Dai et al. report that DeepSeekMoE 2B achieves performance comparable to GShard 2.9B despite the latter having approximately 1.5 times the expert parameters and computation. They also report that DeepSeekMoE 16B achieves performance comparable to LLaMA2 7B using about 40% of the computation in their reported comparison. These are empirical results for the configurations and evaluations in that paper, rather than general guarantees for every fine-grained MoE design (**dai2024deepseekmoe**).

2.3.8 Expert Structure, Specialization, and Practical Constraints

The presence of multiple experts does not by itself establish that they have learned distinct functions. Experts may acquire overlapping parameters or exhibit similar output behavior. Lo et al. investigate this issue by analyzing the projection matrices $\mathbf{W}_{\text{up}}$, $\mathbf{W}_{\text{act}}$, and $\mathbf{W}_{\text{down}}$ in several MoE models (**lo2025closer**).

One way to compare two parameter matrices is to flatten them into vectors and calculate cosine similarity. For nonzero vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, cosine similarity is

$$\text{CosSim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}, \tag{2.130}$$

where

$$\|\mathbf{a}\|_2 = \sqrt{\sum_{j=1}^{n} a_j^2}$$

is the Euclidean norm. Cosine similarity measures angular alignment and is invariant to positive rescaling of either vector. It does not, however, measure equality of magnitudes or guarantee equality of the functions implemented by the corresponding networks.

Flattened-matrix comparison is also sensitive to neuron ordering. Two experts may contain similar units in different orders, while a direct element-by-element comparison pairs units that do not correspond. Lo et al. therefore investigate neuron-level comparisons, including a reordering procedure based on the Jonker–Volgenant algorithm for a linear assignment problem. Their analysis treats a gated FFN neuron as the combination of the corresponding rows of $\mathbf{W}_{\text{up}}$ and $\mathbf{W}_{\text{act}}$, together with the corresponding column of $\mathbf{W}_{\text{down}}$. This connects the neuron decomposition in equation (2.97) to a more detailed analysis of expert similarity (**lo2025closer**).

In their investigated models, Lo et al. report that matrix-level similarities differ across architectures and layers. For Mixtral, they report cosine similarities primarily in the range 0.2–0.4 between experts, while similarities between Mixtral experts and the dense Mistral FFN are approximately 0.6 in the examined comparisons. They also report that similarities between the routed experts of DeepSeekMoE and the experts of Grok-1 are close to zero in their analysis. These observations are specific to the models, layers, matrices, and comparison procedures studied. They should not be interpreted as a universal numerical measure of expert specialization.

The choice of expert count, expert dimension, number of active experts, and frequency of MoE layers also affects the architecture. Cai et al. identify these as important design variables and describe configurations in which MoE layers replace every second FFN layer, every fourth FFN layer, or the FFN in every layer. A larger number of experts can increase total parameter capacity, while a larger expert dimension increases the capacity and cost of each expert. Increasing the number of active experts changes the computation performed per token. These choices must therefore be considered together rather than optimized independently (**cai2025survey**).

The theoretical arithmetic savings of sparse FFNs do not automatically translate into proportional reductions in wall-clock latency. Routing requires score calculation and expert selection. The implementation must also dispatch tokens to the selected experts and combine their outputs. When experts are distributed across accelerators, this can introduce communication and synchronization costs. Zhang et al. review practical issues

including irregular memory access, cross-device communication, fragmented workloads, and hardware underutilization. These system effects can reduce the practical benefit of sparse computation, particularly under latency or memory constraints (**zhang2025moe**).

The architecture must consequently balance representational capacity, expert specialization, routing quality, load distribution, memory requirements, and the communication pattern imposed by the hardware. An MoE layer is not efficient merely because it activates a small fraction of its parameters; its routing and execution mechanisms must also use the available computational resources effectively.

2.3.9 Limitations and Connection to Activation Functions

The development of FFN architectures reveals a progression from a shared nonlinear transformation applied to every token, through gated intermediate representations, to conditional computation over multiple FFN experts. The conventional FFN provides a position-wise mapping whose parameter count and arithmetic cost are determined by its input, intermediate, and output dimensions. Gated FFNs introduce an additional learned projection that modulates intermediate features. MoE architectures extend the same basic feed-forward computation by selecting a subset of expert networks for each token.

These architectures introduce different trade-offs. Increasing the intermediate dimension increases the capacity and cost of a dense FFN. Adding a gate projection increases the parameterization and computation of a gated FFN. Increasing the number of MoE experts increases total expert capacity, but sparse execution introduces routing, capacity, load-balancing, and communication constraints. Fine-grained segmentation and shared expert isolation provide strategies for organizing expert parameters, but they do not remove the need to evaluate the quality of specialization or the practical cost of routing.

The mathematical role of the activation function remains central throughout this progression. In a conventional FFN, it supplies the nonlinearity between the two projections. In a gated FFN, an activation function transforms the control projection before it is combined with the up-projection. In an MoE architecture, the router also uses a probability transformation to produce expert-selection weights. These operations may all involve nonlinear functions, but they act on different representations and serve different architectural purposes. The next section examines activation functions as mathematical operators, establishing the properties needed to understand their effects on feature transformations, gated computation, and gradient-based learning.

2.4 Activation Functions

2.5 Normalization

2.6 Attention Mechanisms

Part II

Optimization and Scaling

3. Optimization

3.1 Optimization Algorithms

3.2 Loss Functions

3.3 Quantization

3.4 Knowledge Distillation

3.5 Parameter-Efficient Fine-Tuning

4. Scaling Laws

Part III

Reinforcement Learning and Neuroscience

5. Reinforcement Learning

5.1 Theoretical Foundations

5.2 Actor–Critic Methods

5.3 Reinforcement Learning with Verifiable Rewards

5.4 Group Relative Policy Optimization for Verifiable Rewards

6. Neuroscience and Cognitive Foundations

6.1 Introduction

6.2 From System 1 to System 2

Part IV

Data and Training

7. Training Data

7.1 Pre-Training Data

7.2 Post-Training Data

8. Training Methods

8.1 Pre-Training

8.2 Post-Training

8.3 Machine Unlearning

Part V

Evaluation, Impact, Philosophy, and Challenges

9. Evaluation

10. Environmental Impacts

11. Philosophy

12. Challenges